

Ames Housing Project — Process & Insights

Sep 26, 2026

1. Project Overview & Goal

This is a group machine learning regression project on the Ames Housing dataset (Ames, Iowa residential sales). The goal: predict a house's `SalePrice` from its physical and locational characteristics, using three required model types — Support Vector Regression (SVR), Linear Regression, and K-Nearest Neighbors (KNN) — and compare them to pick a final winner.

Work was split by section across teammates and merged into one notebook: EDA, preprocessing, feature engineering, and the SVR model on one side; Linear Regression and KNN on another. A significant part of the final effort went into verifying the merged pipeline was internally consistent (see Section 7) before final evaluation.

2. Dataset Understanding & EDA

The Ames Housing dataset has ~2,900 rows and ~80 raw columns describing each home (lot, quality ratings, room counts, garage, basement, sale conditions, etc.), with `SalePrice` as the target.

Missing values: a data-quality pass flagged which columns had missing data and why — most "missing" categorical values actually meant "feature not present" (e.g. no pool, no basement) rather than truly unknown data.

Target analysis: `SalePrice` is right-skewed (a long tail of expensive homes). Comparing raw vs. `log1p`-transformed distributions showed the log transform substantially reduces skew and kurtosis — this insight later justified log-transforming the target for Linear Regression.

Numerical & categorical analysis: boxplots across all numerical features screened for outliers; categorical features were examined against `SalePrice` to see which categories (e.g. neighborhood, exterior quality) separate price the most.

Correlation analysis: numeric features were ranked by correlation with `SalePrice`, surfacing the strongest linear predictors (overall quality, living area, garage capacity, etc.) ahead of feature engineering.

Automated EDA: a `ydata-profiling` report was generated as a cross-check against the manual EDA, catching anything the manual pass might have missed (near-duplicate columns, unusual distributions, high-cardinality categoricals).

3. Data Preprocessing Pipeline

Duplicates: exact full-row duplicates were dropped from the raw dataset *before* the train/val/test split — doing this first prevents the same physical record ending up in two different splits.

Split: a 60/20/20 train/validation/test split (`random_state=42`), done immediately after de-duplication and *before* any statistic (a median, a mode, a category threshold) was computed from the data. This ordering is the backbone of the whole leakage-prevention strategy: everything downstream is learned from `train_df` only, then applied to `val_df` / `test_df`.

Missing values: learned entirely from training data — Lot Frontage filled with the *training-set* per-neighborhood median (falling back to a global training median for unseen neighborhoods), Masonry Veneer Type and Electrical filled with their *training-set* mode. The same fitted values were applied unchanged to validation and test.

Why this mattered: the very first version of this pipeline computed these statistics on the *full* dataset before splitting — a classic data-leakage bug, since it let information from validation/test rows quietly influence values used to train the model. This was caught and fixed by moving the split earlier and rewriting every imputation/encoding step to fit on `train_df` only (see Section 7 for how this was verified across the whole merged notebook).

4. Feature Engineering & Selection

New features (11 total): House Age, Years Since Remodel, Total SF (combined basement + 1st + 2nd floor square footage), Total Bathrooms (full + half, including basement), Total Porch SF, and binary flags (Has Pool, Has Garage, Has Fireplace, Has 2nd Floor, Is Remodeled, Is New House). These consolidate raw measurements into the kind of composite variables that tend to correlate more directly with price than their individual components.

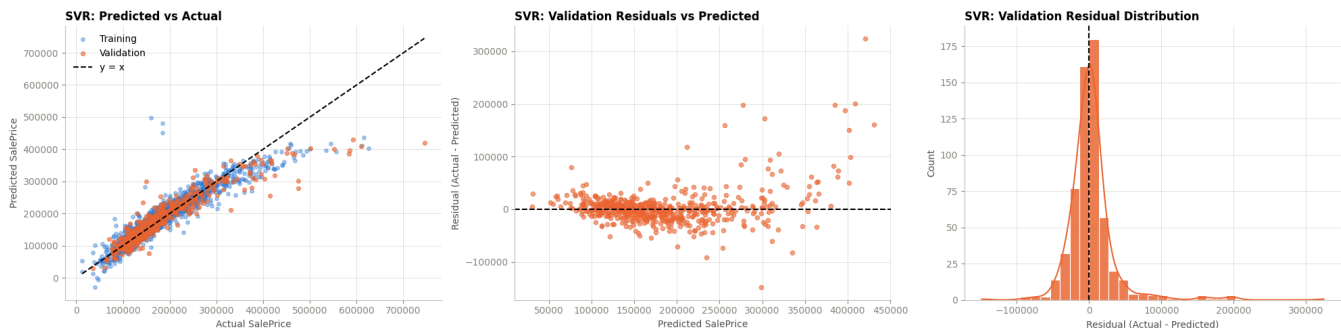
Feature selection: `Garage Area` was dropped after confirming it was highly correlated with `Garage Cars` (both measuring garage capacity) while carrying no extra predictive signal — keeping both would have introduced multicollinearity without benefit.

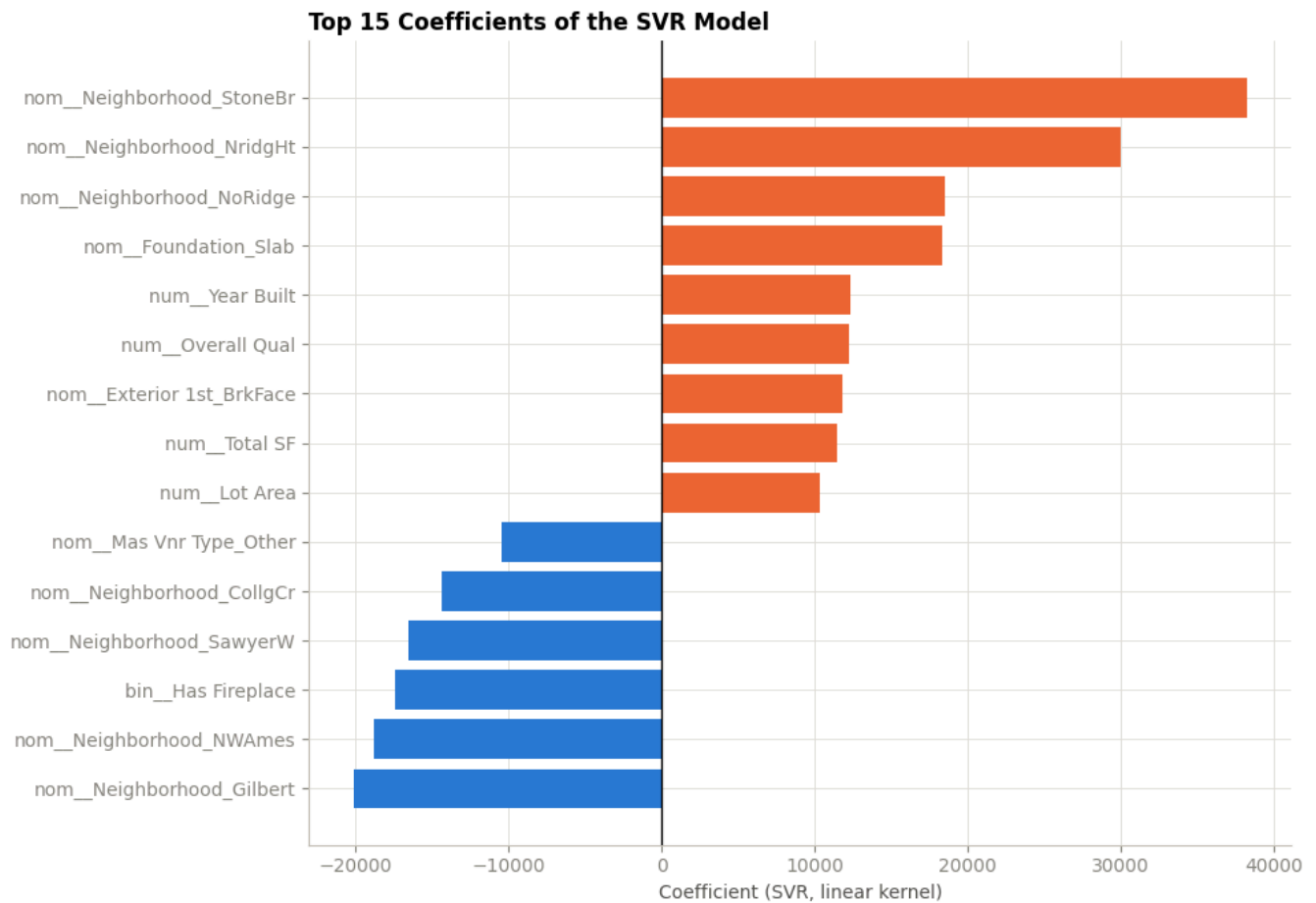
Encoding & scaling (inside the pipeline, not before it): nominal categoricals were one-hot encoded, ordinal categoricals (quality/condition ratings) were ordinal-encoded using explicit worst-to-best orderings, and continuous numeric features went through a Yeo-Johnson power transform (handles skew automatically, works with negative/zero values, and standardizes in the same step). Rare categories (fewer than 30 training occurrences) were grouped into "Other" — with the threshold learned from training data only. Binary flag columns were passed through untouched. All of this was wrapped in a single `ColumnTransformer` inside each model's `Pipeline`, so it refits correctly within cross-validation instead of being computed once on the whole dataset.

5. Modeling: SVR, Linear Regression, KNN

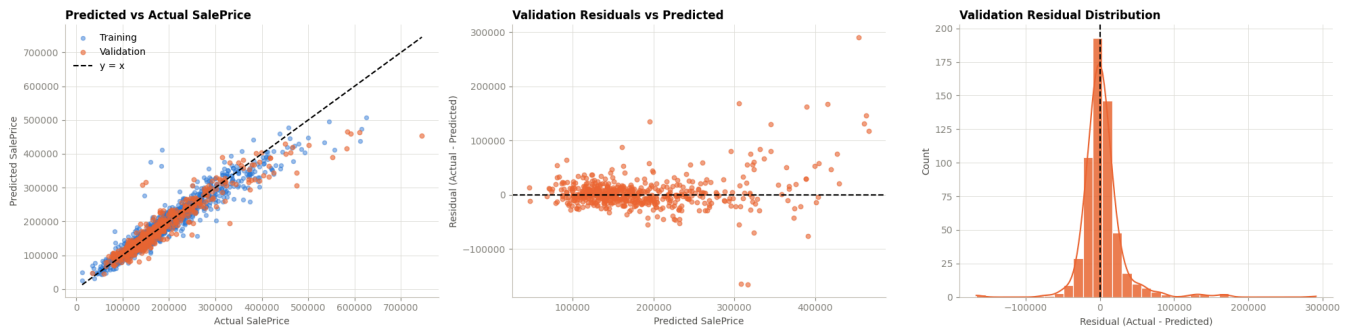
All three models share the exact same preprocessing pipeline (`ColumnTransformer`), each wrapped inside its own `Pipeline` and tuned with `GridSearchCV` (5-fold cross-validation) so preprocessing statistics are refit correctly within every fold.

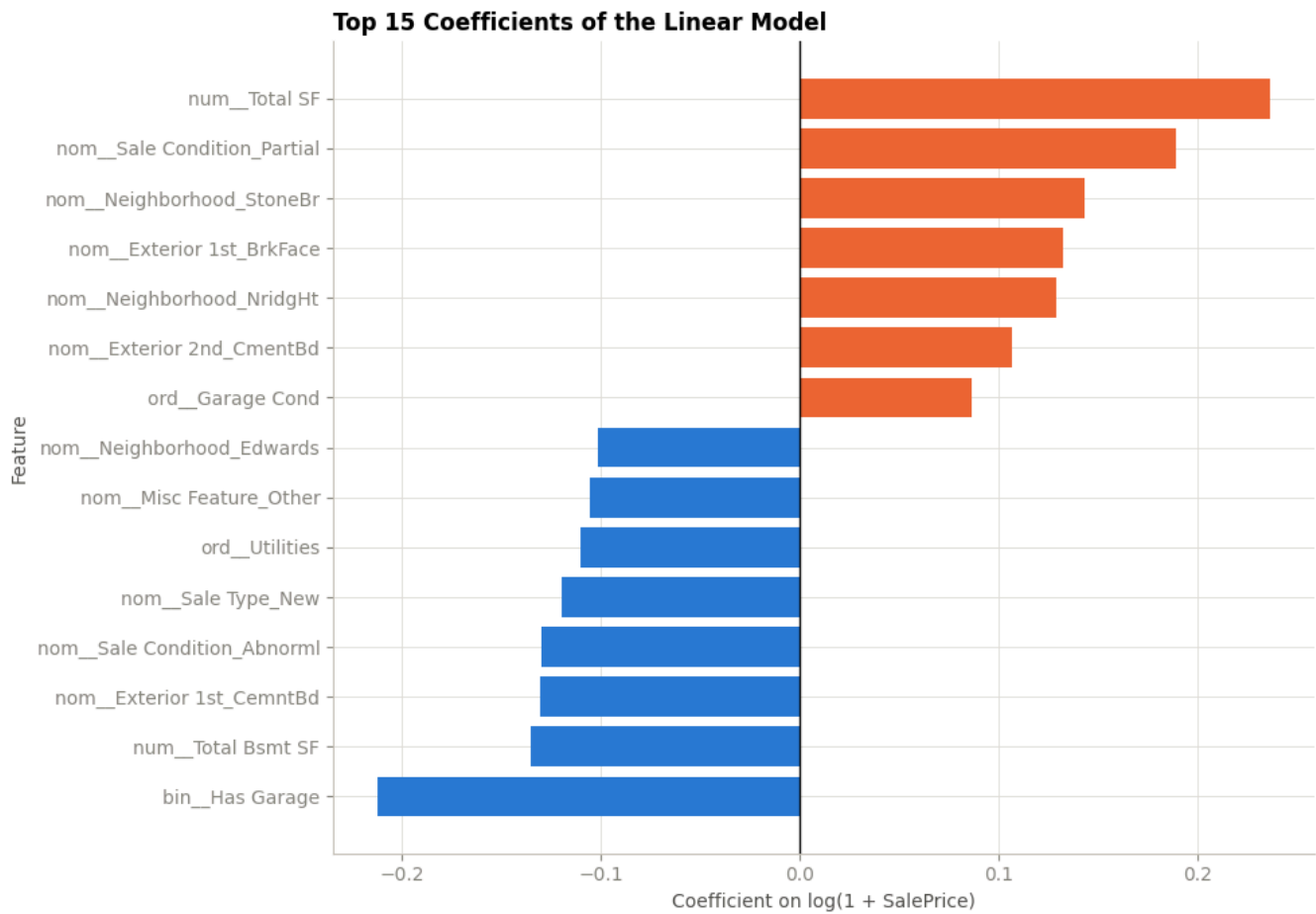
SVR: linear kernel selected by the grid search over `C` and `epsilon`. Best parameters: `C=5000`, `epsilon=0.005`, CV RMSE $\approx$ \$29,748, training $R^2 \approx 0.874$.



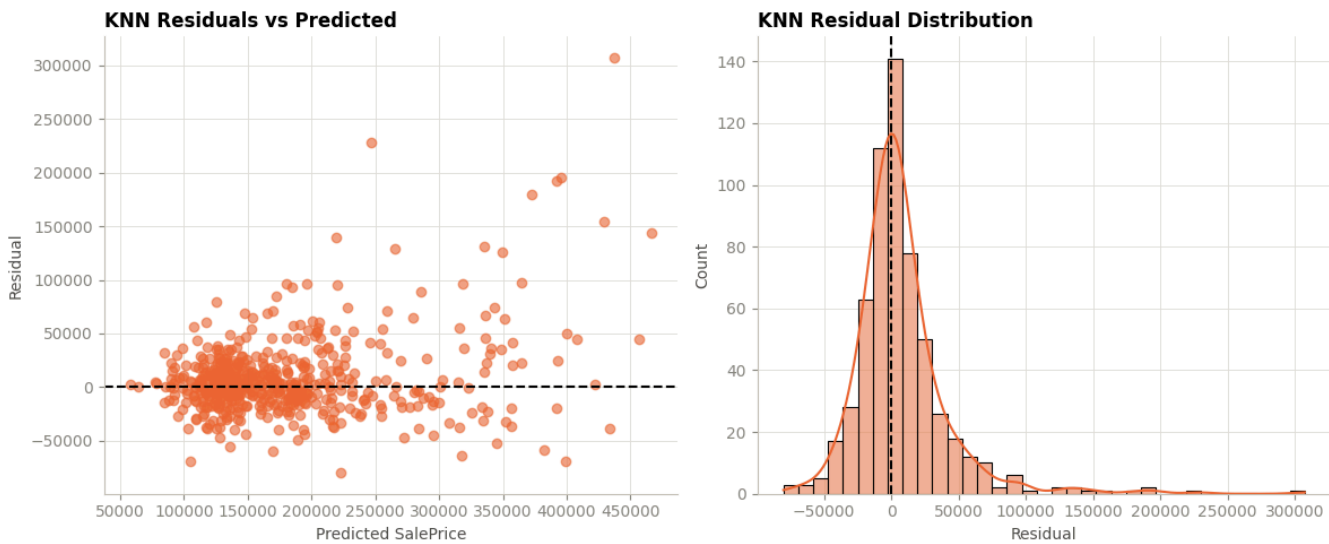


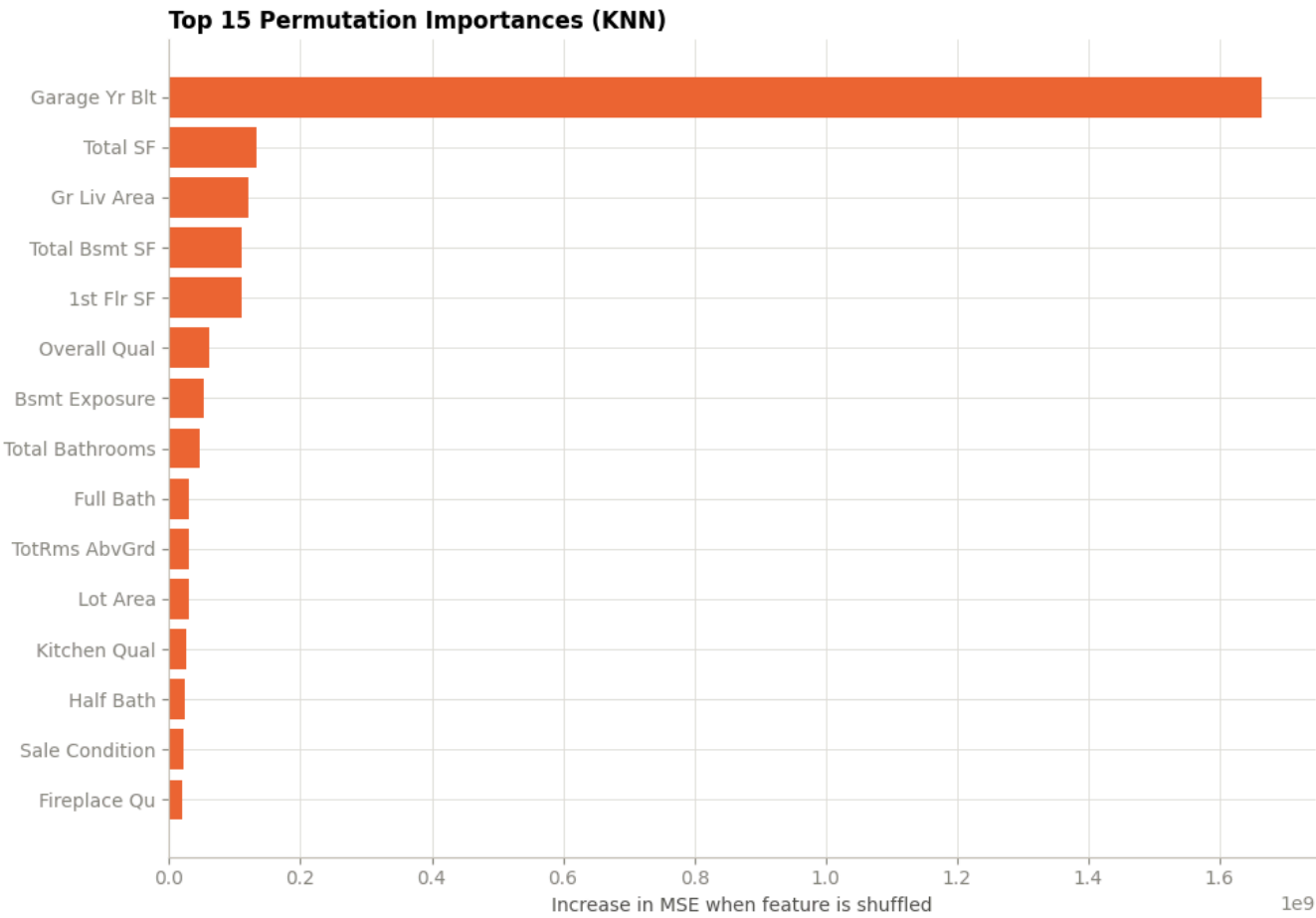
Linear Regression: fit with a log-transformed target (`TransformedTargetRegressor` with `log1p / expm1`) so its coefficients sit on a more normally-distributed scale, which the EDA in Section 2 had already flagged as beneficial. Ridge and Lasso variants were also grid-searched as a sanity check on regularization; the plain log-target linear model was ultimately used since regularization didn't meaningfully improve it.





KNN: tuned over number of neighbors, distance weighting, and distance metric (Manhattan vs. Euclidean). Best parameters: `n_neighbors=5`, `weights='distance'`, `p=1` (Manhattan distance).

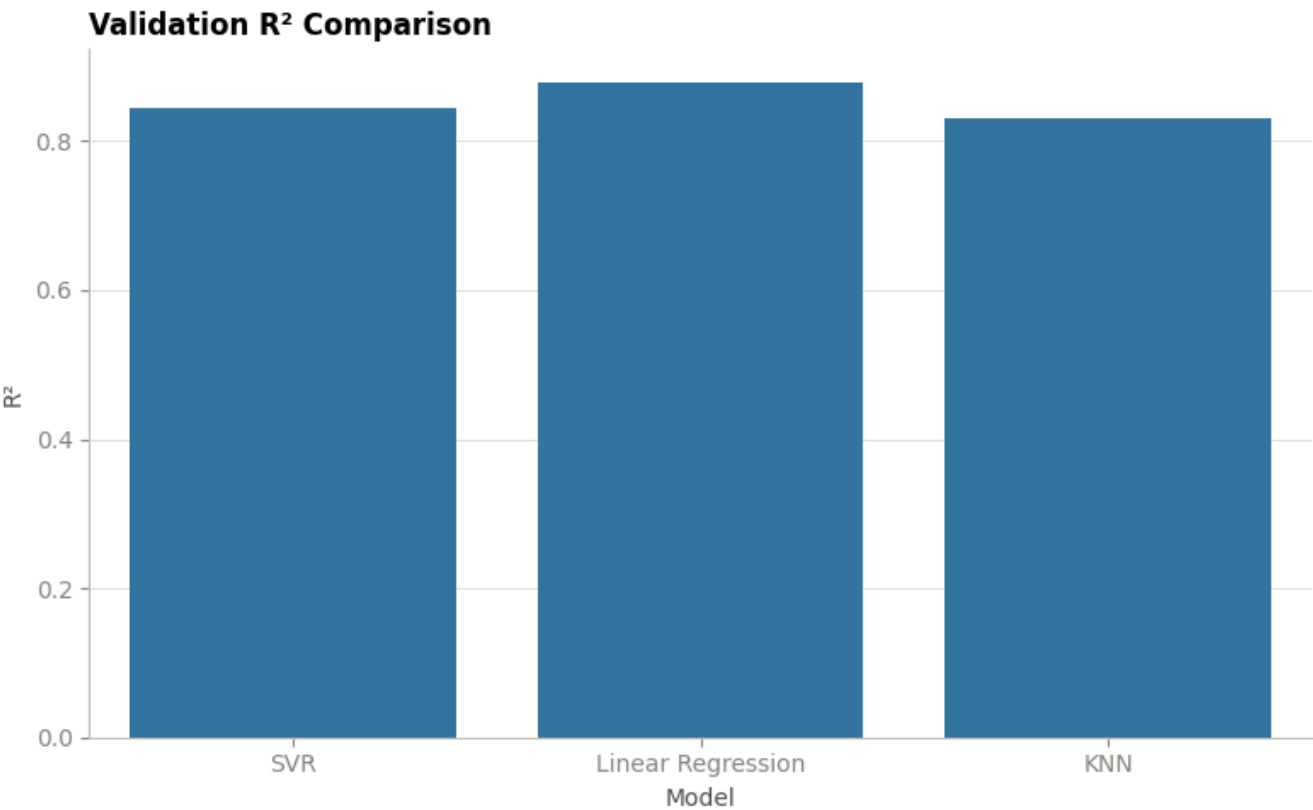
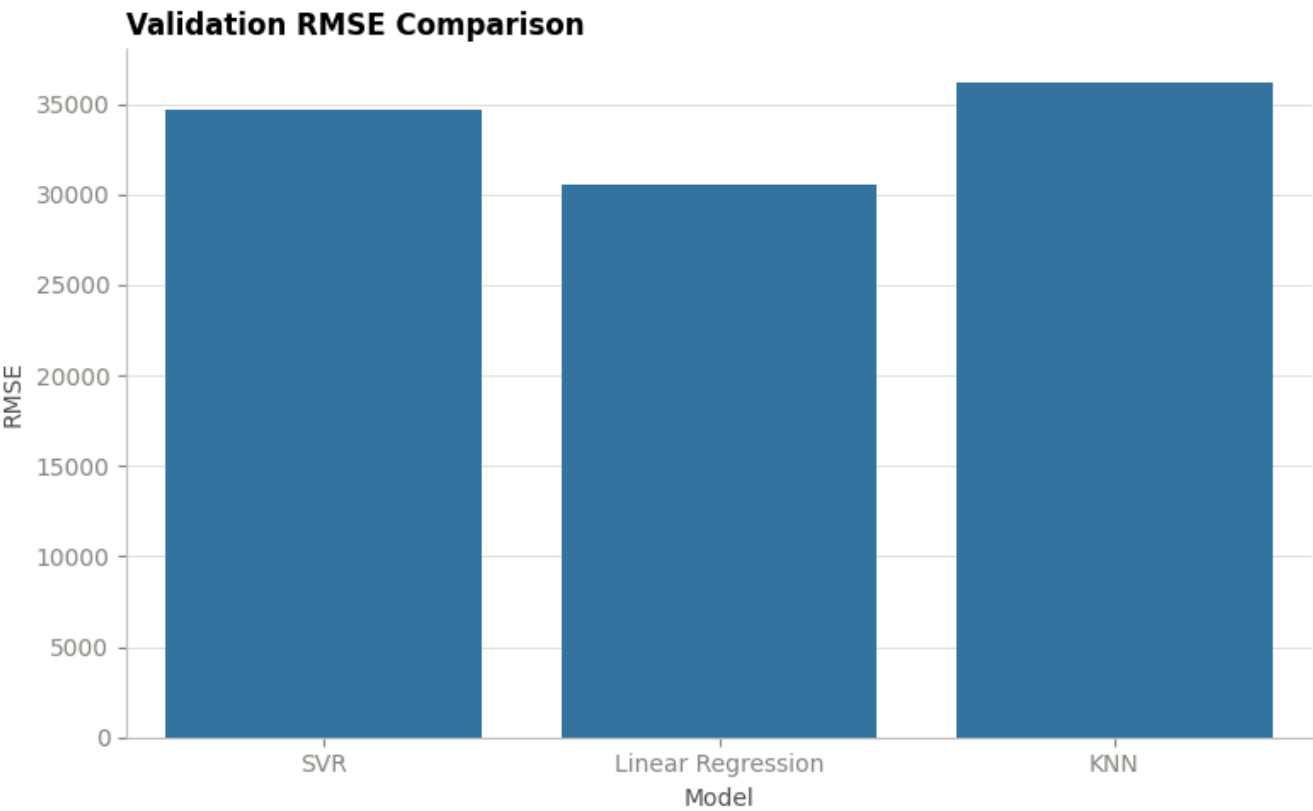


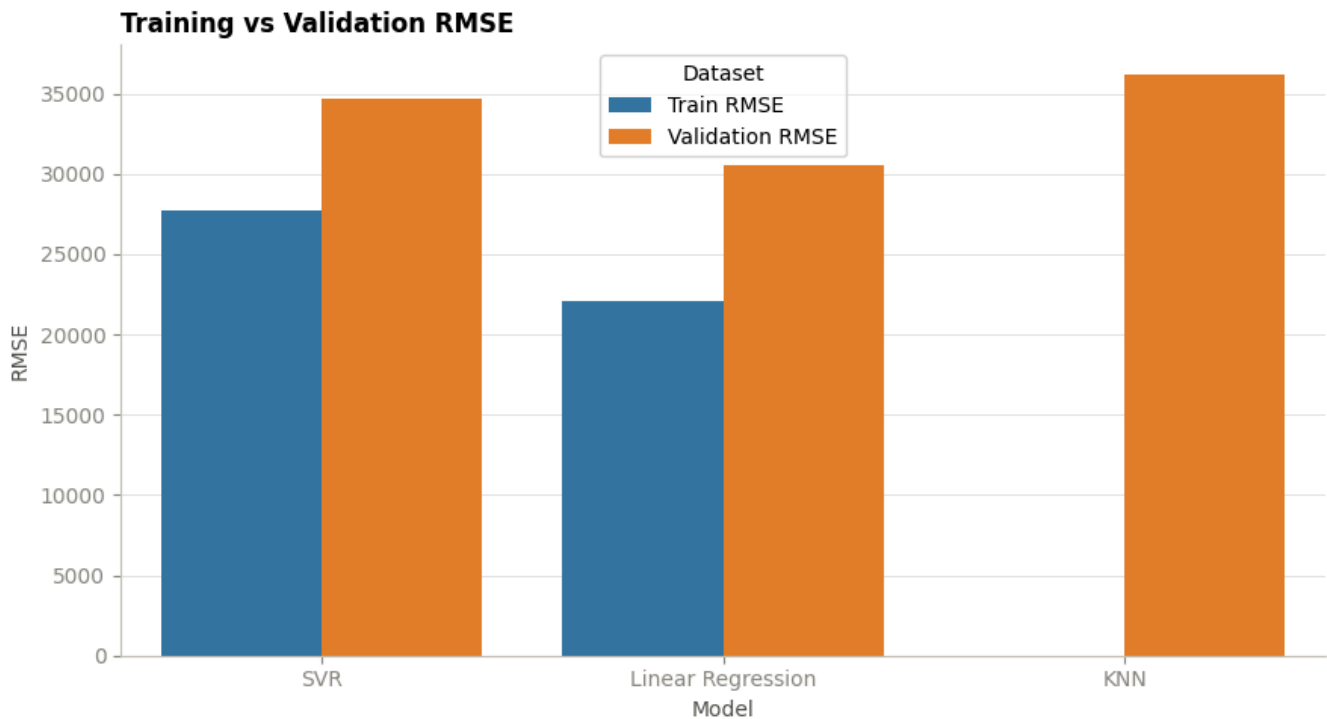


6. Evaluation & Results

Validation comparison:

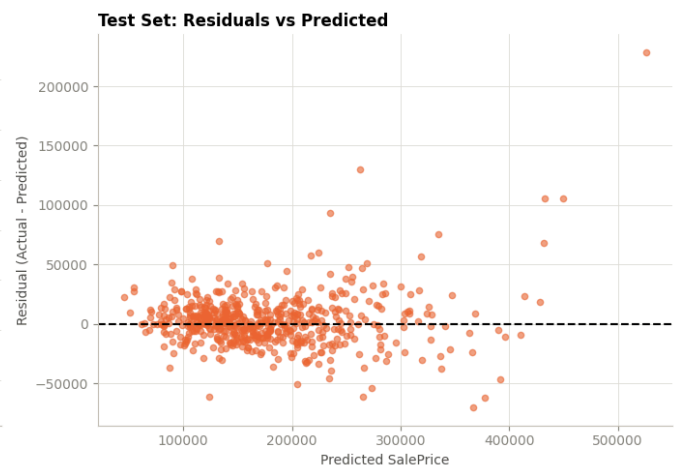
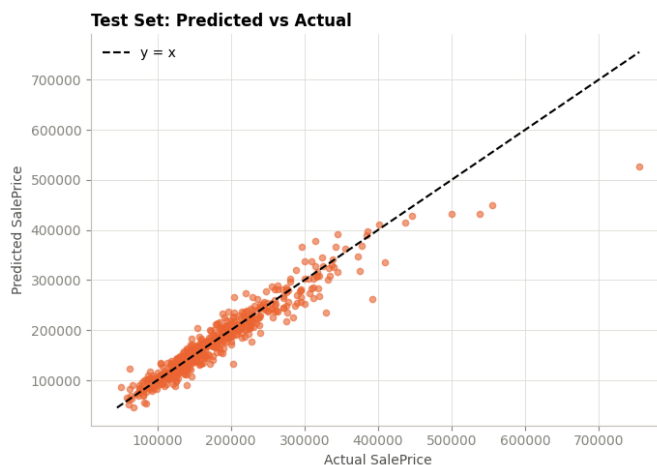
Model	Validation MAE	Validation RMSE	Validation R ²
Linear Regression	\$16,917	\$30,495	0.879
SVR	\$19,026	\$34,649	0.844
KNN	\$21,746	\$36,221	0.830





Winner: Linear Regression, meaningfully ahead of both SVR and KNN on this run — about 12% lower validation RMSE than SVR and 16% lower than KNN.

Final test-set evaluation (winning model only, run once as the final check): RMSE actually came in *lower* on test than validation this run — \$22,374 ($R^2 = 0.915$), down from \$30,495 on validation. That's the reverse of the more typical pattern, and it comes from the same root cause documented in Section 7: with a right-skewed target and ~586-row validation/test splits, RMSE is highly sensitive to exactly which high-value homes land in which split. The single largest miss this run was a \$755,000 home underpredicted at \$526,223 (a \$228,777 error) — see the residual plot below. The takeaway to trust is the models' *relative* ranking (Linear Regression < SVR < KNN on both splits), not the exact RMSE value attached to any one split.



7. Problems We Faced & How We Solved Them

1. Data leakage in the original preprocessing order. The first version of the pipeline computed imputation medians/modes and rare-category thresholds on the *full* dataset before splitting into train/val/test — meaning validation and test information was leaking into values used during training. **Fix:** moved the train/val/test split to right after duplicate removal, and rewrote every statistic-dependent step (missing-value fills, rare-category grouping) to fit exclusively on `train_df`, then apply that fitted rule to val/test.

2. Verifying a merged notebook stayed internally consistent. The notebook combines each teammate's section (EDA, preprocessing, and SVR from one contributor; Linear Regression and KNN from others). Before finalizing, every model's pipeline was checked to confirm it used the *same* fitted `ColumnTransformer` object (via `clone(preprocessor)`) rather than an independently-built one — catching one inconsistency where SVR referenced the shared preprocessor directly instead of cloning it. Functionally harmless (`GridSearchCV` clones internally before fitting), but fixed for consistency and to remove a fragile dependency on that internal behavior.

3. A misleading KNN training metric. KNN's best parameters included `weights='distance'`, which makes a model evaluated on its own training set trivially "predict" each point perfectly (its own nearest neighbor is itself, at distance 0) — producing Train $R^2 = 1.0$ and Train RMSE = 0. This looked like a bug at first glance. **Resolution:** confirmed it's expected behavior for distance-weighted KNN (not a leak or an error), and documented it directly in the notebook so it doesn't get mistaken for one — validation/test metrics are the only ones treated as meaningful for this model.

4. RMSE varying more than expected between train, validation, and test. Both SVR and Linear Regression showed *lower* RMSE on validation than on training — the reverse of what's normally expected. Investigating this traced it to `SalePrice` being right-skewed: RMSE is highly sensitive to which specific high-value homes land in which split, since large errors are squared before averaging. This was documented as a known property of the metric and the data, rather than treated as a modeling mistake.

5. Local environment issues. A PowerShell script-execution policy blocked activating the project's virtual environment (`Activate.ps1` refused to run as unsigned); resolved by scoping `Set-ExecutionPolicy` to the current process only. Separately, a teammate's environment had a newer NumPy (2.4) that broke `ydata-profiling`'s dependency on `numba`, which caps out at NumPy 2.3; resolved by pinning `numpy<2.4` in `requirements.txt` so the same environment reproduces for everyone rather than depending on install timing.

8. Key Takeaways & Limitations

Why Linear Regression won: after Yeo-Johnson normalizing the skewed numeric features and log-transforming the target, the remaining relationship between features and price is close to linear — Ames Housing prices are driven largely by additive factors (square footage, quality ratings, neighborhood). SVR's grid search also converged on a *linear* kernel, landing in a similar place. KNN struggled most, likely due to the curse of dimensionality once one-hot encoding pushed the feature space past 100+ dimensions — distance-based methods degrade as dimensionality grows.

Limitations: RMSE is sensitive to which specific high-value homes land in each random split, which is why train/validation/test RMSE didn't move in the direction usually expected. A stratified split by price quantile (ensuring each split gets a proportional mix of cheap/mid/expensive homes) would make these comparisons more stable in a future iteration.

Process lessons: (1) always fit preprocessing statistics after splitting, never before; (2) when merging pipelines across teammates, explicitly verify every model uses the same underlying fitted preprocessing object; (3) an unexpected metric (like KNN's perfect training fit) is worth investigating before assuming it's either a bug or something to hide — sometimes it's neither; (4) pin dependency versions in `requirements.txt` rather than leaving them open, so the same code behaves the same way on every machine that runs it.